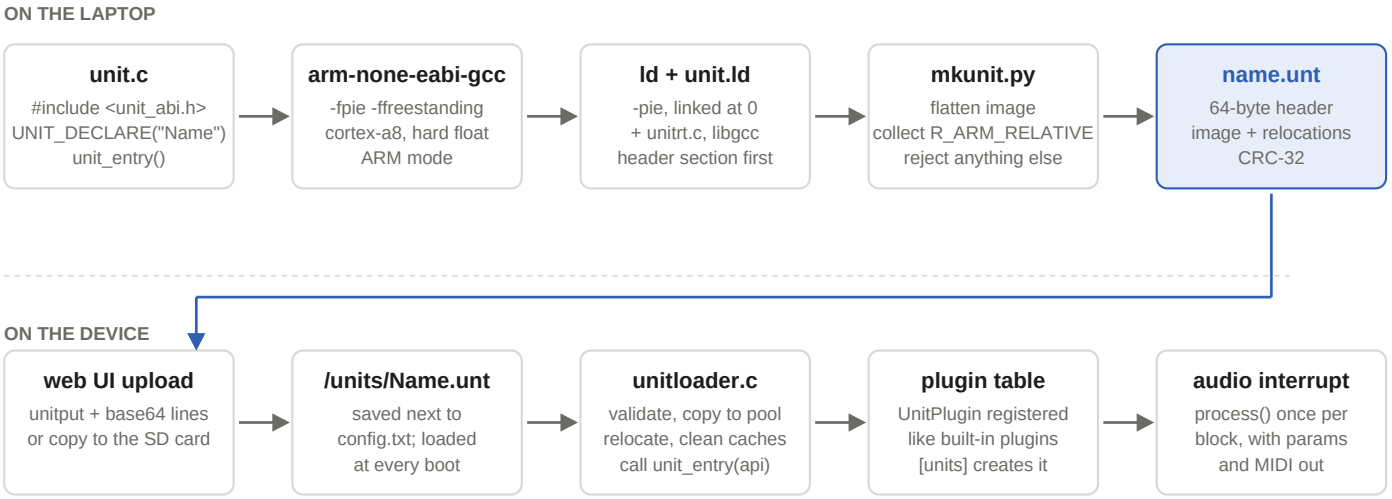
Metalhead unit ABI

Version 1 · loadable units for the metalhead firmware · 11 October 2026

A **unit** is an audio/MIDI plugin built on the laptop, separately from the firmware, and loaded by the running device. It is uploaded from the web UI (or copied to the SD card), stored as `/units/<Name>.unt` next to `config.txt`, loaded at every boot and then used in `[units]` like a built-in plugin. This document summarises the binary interface between the two: what a unit file contains, how it is loaded, which functions run where, and the exact layout of every structure. The source of truth is `ccs/units/include/unit_abi.h`; offsets and sizes below come from the ARM compiler.

ABI VERSION **1** exact match required	TARGET **Cortex-A8, ARM mode** AAPCS, hard float, NEON	CODE **Position independent** -fpie, freestanding, libgcc only
RELOCATIONS **R_ARM_RELATIVE only** base address added per word	HOST SERVICES **5 functions** log, allocate, 3 file reads	PLUGIN INTERFACE **36-byte vtable** identical to the firmware's

From source to sound



There is no dynamic linker. A unit cannot call firmware or C library functions by name: everything it needs from the host comes through two tables of function pointers, **UnitHostApi** (passed once to unit_entry) and **UnitProcessHost** (passed to every process call). mkunit.py fails the build if the unit references anything outside its own image, so a missing symbol never reaches the device.

Lifecycle and calling contexts

The loader validates the file, copies the image into a 2 MB code pool in DDR, zeroes its bss, adds the load address to every relocated word, cleans the D-cache, invalidates the I-cache and branch predictor, calls unit_entry and registers the returned plugin. Units are never unloaded.

Call	When	Runs in	Allowed
`unit_entry(api)`	once, at boot (before config.txt) or right after an upload	main loop	everything: log, allocate, file access, slow set-up work (e.g. loading samples)
`createInstance()`	for each [units] line when config.txt is loaded	main loop	log, allocate, file access; return NULL when out of instances
`getName, getNum*, getParameter*`	while loading the unit and resolving config.txt	main loop	must be pure and return the same answers every time
`process(...)`	once per audio block (one USB frame of samples)	**audio interrupt**	log and sendMIDIEvent only. No file access (refused), no allocate, no blocking or waiting

process() signature

```
void process(void * instance, UnitProcessHost * host,
             size_t nrInChannels,  float ** inSamples,    // unconnected inputs contain silence
             size_t nrOutChannels, float ** outSamples,   // outputs start zeroed; add into them
             size_t frameCount,                           // samples per channel in this block
             UnitParameterChange * parameterChanges, size_t parameterChangeCount);
```

Things that bite. Parameter ids must be non-zero (0 means "no such parameter" to the firmware). UnitParameterChange is 12 bytes, not 8: it is a packed union of an unpacked struct, so *value* sits at offset 8. At most 16 MIDI events are delivered per block for all units together. Floating point and NEON are fine in process(): the IRQ handler saves the full VFP/NEON state.

Structures

All structures are little endian with natural ARM EABI alignment unless noted. The firmware static-asserts that the size and every field offset match its own plugin structures (`unitloader.c`), so a unit's UnitPlugin goes straight into the plugin table.

UnitPlugin 36 bytes

Returned by unit_entry. Every function pointer must point into the unit's own image (the loader checks).

Offset	Size	Field	Type	Meaning
0	4	`vendorId`	`uint32_t`	free for the unit; not used by the firmware
4	4	`getName`	`void (*)(char *name, size_t max)`	type name used in [units]; must equal the name in UNIT_DECLARE
8	4	`createInstance`	`void *(*)(void)`	one per [units] line; NULL if none left. Instances are never destroyed
12	4	`getNumInputs`	`uint8_t (*)(void)`	audio inputs unit:1..N
16	4	`getNumOutputs`	`uint8_t (*)(void)`	audio outputs unit:1..N
20	4	`getNumParameters`	`uint8_t (*)(void)`	parameters reachable from [midimap]
24	4	`getParameterId`	`uint16_t (*)(size_t index)`	stable id, **must be non-zero**; arrives in UnitParameterChange
28	4	`getParameterName`	`void (*)(size_t, char *, size_t)`	name used in [midimap] (unit:name)
32	4	`process`	`void (*)(...)`	once per audio block, in the audio interrupt (see below)

UnitProcessHost 64 bytes

Passed to every process() call; only valid during that call.

Offset	Size	Field	Type	Meaning
0	4	`play`	`void (*)(void)`	start the transport
4	4	`stop`	`void (*)(void)`	stop the transport
8	4	`requestTempo`	`void (*)(float bpm)`	change the tempo
12	4	`sendMIDIEvent`	`void (*)(hostData, UnitMidiPacket)`	queue MIDI on the unit's output ([midiroutes]); max 16 per block, all units together
16	40	`timeInfo`	`UnitTimeInfo`	timing for this block
56	4	`hostData`	`struct UnitHostData *`	opaque; pass to sendMIDIEvent

UnitTimeInfo 40 bytes

Embedded in UnitProcessHost at offset 16.

Offset	Size	Field	Type	Meaning
0	8	`microSeconds`	`uint64_t`	device clock at the start of the block
8	4	`sampleRate`	`float`	Hz (48000)
12	4	`isPlaying`	`int`	transport running (Ableton Link / MIDI clock timeline)
16	8	`barStartPos`	`double`	start of the current bar, in quarter notes
24	8	`ppqPos`	`double`	position in quarter notes
32	4	`tempo`	`float`	BPM
36	4	`timeSigNumerator`	`int`	quantum (beats per bar)

UnitParameterChange 12 bytes

Produced from [midimap]: a CC or note on a device mapped to unit:parameter.

Offset	Size	Field	Type	Meaning
0	1	`unitIndex`	`uint8_t`	index of the receiving unit
2	2	`parameterId`	`uint16_t`	as returned by getParameterId()
4	1	`midiValue`	`uint8_t`	raw 0..127 (CC value or note velocity)
8	4	`value`	`float`	midiValue / 127

UnitMidiPacket 4 bytes

A USB-MIDI event packet. SysEx is sent as CIN 0x4 packets followed by a 0x5-0x7 end packet.

Offset	Size	Field	Type	Meaning
0	1	`header`	`uint8_t`	(cable << 4) | code index number; cable 0 = default jack
1	1	`midi0`	`uint8_t`	status byte (e.g. 0x90 note on, ch 1)
2	1	`midi1`	`uint8_t`	data 1
3	1	`midi2`	`uint8_t`	data 2

UnitHostApi 28 bytes

Passed to unit_entry; stays valid and may be kept. Fields marked v1+ were appended to ABI 1 after its first release: check `UNIT_HOST_HAS(api, field)` before calling them, so the unit still loads on older firmware.

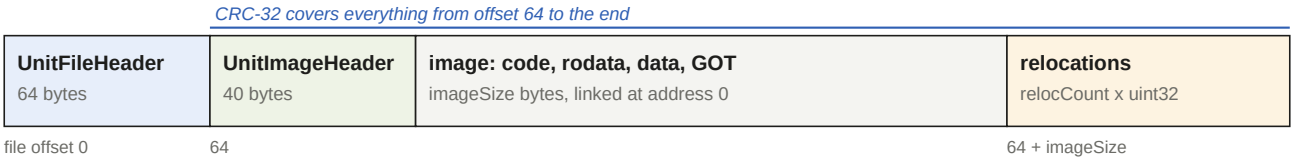
Since	Offset	Size	Field	Type	Meaning
v1	0	4	`abiVersion`	`uint32_t`	UNIT_ABI_VERSION of the firmware
v1	4	4	`size`	`uint32_t`	sizeof(UnitHostApi) in the firmware
v1	8	4	`log`	`void (*)(const char *fmt, ...)`	queued console log; safe from process()
v1+	12	4	`allocate`	`void *(*)(size_t)`	memory for the unit's life, 8-byte aligned, not zeroed; NULL when the 64 MB heap is full
v1+	16	4	`fileSize`	`int32_t (*)(const char *path)`	size of a file, < 0 if missing or a directory
v1+	20	4	`readFile`	`int32_t (*)(path, offset, buf, len)`	bytes read, < 0 on error
v1+	24	4	`readDirectory`	`int32_t (*)(path, index, entry *)`	1 = entry filled, 0 = no more, < 0 = error

UnitDirectoryEntry 72 bytes

Filled by readDirectory. Entries are returned in on-disk order; sort if order matters.

Offset	Size	Field	Type	Meaning
0	64	`name`	`char[64]`	file or folder name (longer names are skipped)
64	4	`size`	`uint32_t`	file size in bytes
68	1	`isDirectory`	`uint8_t`	non-zero for folders

The .unt file



The image is linked at address 0 and starts with the UnitImageHeader that UNIT_DECLARE places in the .unit_header section. After loading at address *base*, the loader adds *base* to the 32-bit word at each relocation offset. The firmware allocates imageSize + bssSize bytes, 64-byte aligned.

UnitFileHeader 64 bytes

Written by mkunit.py.

Offset	Size	Field	Type	Meaning
0	4	`magic`	`uint32_t`	"UNT1" (0x31544E55)
4	2	`abiVersion`	`uint16_t`	must equal the firmware's UNIT_ABI_VERSION
6	2	`headerSize`	`uint16_t`	64
8	4	`imageSize`	`uint32_t`	bytes of image that follow, a multiple of 4
12	4	`bssSize`	`uint32_t`	zeroed bytes the loader adds after the image
16	4	`relocCount`	`uint32_t`	uint32 offsets after the image
20	4	`entryOffset`	`uint32_t`	offset of unit_entry in the image
24	4	`crc32`	`uint32_t`	IEEE CRC-32 (zlib.crc32) of everything after this header
28	4	`reserved`	`uint32_t`	0
32	32	`name`	`char[32]`	type name, NUL-terminated, [A-Za-z0-9_]{1,31}

UnitImageHeader 40 bytes

First bytes of the image; checked against the file header.

Offset	Size	Field	Type	Meaning
0	4	`magic`	`uint32_t`	"UNIT" (0x54494E55)
4	4	`abiVersion`	`uint32_t`	UNIT_ABI_VERSION the unit was built with
8	32	`name`	`char[32]`	type name; written by UNIT_DECLARE("Name")

What the loader checks

1	magic "UNT1", headerSize 64 and an exact ABI version match
2	sizes consistent with the file length; image a multiple of 4 bytes
3	CRC-32 over image and relocations
4	entry point and every relocation inside the image, 4-byte aligned
5	image header magic, ABI version and name match the file header
6	after unit_entry: the plugin and all its function pointers lie inside the unit's image
7	getName() returns the name in the file; no built-in plugin or loaded unit has that name

Versioning rules

Change	Version	Effect on existing units
Append a service to the end of UnitHostApi	stays 1	none; new units check UNIT_HOST_HAS before using it
Change, reorder or remove any field; change a calling convention or a meaning	bump UNIT_ABI_VERSION	refused by the loader and the web UI until rebuilt
Change the firmware's plugin structures (plugins.h)	n/a	the firmware stops compiling (static asserts) until the ABI is updated

Limits

Resource	Limit
Unit code and static data	2 MB pool for all units; 256 KB per .unt file
Heap (allocate)	64 MB shared by all units (0x8C000000 to 0x90000000), never freed
Units / plugins	16 loaded units; 32 plugins including the built-in ones
MIDI out	16 events per audio block, all units together
Files	read only, and only from unit_entry / createInstance
Replacing a unit	saved immediately; the new version loads at the next reboot

A minimal unit

```
#include <unit_abi.h>

UNIT_DECLARE("Gain");                                // type name used in [units]

static float gains[4];
static int used;

static void copy(char * d, const char * s, size_t max) {
    size_t i = 0;
    for (; s[i] && i + 1 < max; i++) d[i] = s[i];
    if (max) d[i] = 0;
}

static void getName(char * n, size_t max) { copy(n, __unit_header.name, max); }
static void * createInstance(void) { if (used == 4) return 0; gains[used] = 1.0F; return &gains[used++]; }
static uint8_t numIn(void) { return 1; }
static uint8_t numOut(void) { return 1; }
static uint8_t numParams(void) { return 1; }
static uint16_t paramId(size_t i) { return 1; }                      // never 0
static void paramName(size_t i, char * n, size_t max) { copy(n, "gain", max); }

static void process(void * inst, UnitProcessHost * host, size_t nIn, float ** in,
                    size_t nOut, float ** out, size_t frames,
                    UnitParameterChange * changes, size_t nChanges) {
    float * gain = inst;
    for (size_t c = 0; c < nChanges; c++) *gain = changes[c].midiValue / 127.0F;
    for (size_t i = 0; i < frames; i++) out[0][i] = in[0][i] * *gain;
}

static const UnitPlugin plugin = { .getName = getName, .createInstance = createInstance,
    .getNumInputs = numIn, .getNumOutputs = numOut, .getNumParameters = numParams,
    .getParameterId = paramId, .getParameterName = paramName, .process = process };

const UnitPlugin * unit_entry(const UnitHostApi * api) {
    api->log("Gain unit ready");
    return &plugin;
}
```

Build with a three-line Makefile (`NAME := gain`, `SRCS := gain.c`, `include ../unit.mk`) to get `build/gain.unt`. Then use it in config.txt:

```
[units]
amp Gain
[audioroutes]
.Scarlett_2i2_USB:1 amp:1
amp:1 .device:1
[midimap]
.nanoKONTROL2:cc0 amp:gain
```

Units in the repository: **vumeter** (level meters on a Launchpad Mini MK3, MIDI out) and **drums** (sampler that loads WAV files from /samples/drums with readDirectory/readFile/allocate).

Upload protocol

The web UI talks to the device console over the USB CDC serial port (or the debug UART). Machine-readable replies start with @; the console prints "> " after each command, which the UI uses as the acknowledgement for every line.

Command	Reply
`unitls`	`@unitabi <version>, then @unit <name> <bytes> <loaded|reboot|failed|new> per file in /units`
`unitput <bytes>`	`@unitput ready; then base64 lines (768 characters, a multiple of 4) until unitend`
`unitend`	`@unitput ok <name> <loaded|unchanged|reboot> or @unitput error <reason>`
`unitrm <name>`	`@unitrm ok (the unit stays loaded until the next reboot)`
`cfgabort`	`leaves unitput (or cfgput) mode without saving`